



UNIVERZITET U NIŠU
ELEKTRONSKI FAKULTET



SISTEMI ZA ANALIZU VELIKIH TOKOVA PROSTORNO-VREMENSKIH PODATAKA

Studijsko istraživački rad

Smer: Računarstvo i informatika

Modul: Softversko inženjerstvo

Student:

Anja Tonsa Milovanović, br. ind. 1815

Mentor:

Prof. dr Dragan Stojanović

Niš, jun 2026. godine

Sadržaj

1. Uvod.....	4
2. Big Data i Big Streaming Data	5
2.1. Big Data	5
2.1.1. Prednosti <i>Big Data</i>	5
2.1.2. Izazovi u radu sa Big Data sistemima	6
2.2. Big Streaming Data.....	6
2.2.1. Prednosti Big Streaming Data sistema.....	7
2.2.2. Izazovi u radu sa Big Streaming Data sistemima.....	8
3. Prostorni i prostorno-vremenski podaci.....	9
3.1. Prostorni podaci	9
3.2. Prostorno-vremenski podaci	9
3.3. Obrada podataka tokova prostorno-vremenskih podataka	9
3.3.1 Batch processing	9
3.3.2. Stream processing	9
3.3.3. Poređenje batch i stream processing-a u obradi prostorno-vremenskih podataka	11
3.3.4. Primena stream procesinga na prostorno-vremenske podatke	12
3.4. Izazovi obrade prostorno-vremenskih tokova podataka	13
4. Tehnologije za obradu tokova prostorno-vremenskih podataka	14
4.1. Apache Kafka.....	14
4.2. Apache Spark Streaming.....	15
4.3. Apache Flink.....	15
4.4. Kafka Streams.....	16
4.5. Poređenje tehnologija.....	17
5. Projektovanje sistema za obradu prostorno-vremenskih tokova podataka	19
5.1. Opis problema.....	19
5.2. Arhitektura sistema	20
5.2.1. Izvor podataka - MTA Bus Time GTFS-Realtime feed.....	20
5.2.2. Producer servis.....	20
5.2.3. Apache Kafka.....	21
5.2.4. Apache Flink.....	21
5.2.5. ClickHouse.....	21
5.2.6. Apache Superset.....	21
5.3. Model i struktura podataka	22
5.4. Implementacija analiza	24

5.4.1. Geofencing - detekcija ulaska, izlaska i zadržavanja u zoni	24
5.4.2. Detekcija predugog zadržavanja	27
5.4.3. Sekvence kretanja između zona	27
5.4.4. Agregacija broja vozila po zoni kroz vremenske prozore.....	29
5.5. Vizualizacija rezultata.....	30
5.6. Eksperimentalna evaluacija.....	31
5.6.1. Analiza uticaja paralelizma na performanse sistema	31
5.6.2. Analiza horizontalne skalabilnosti Flink klastera	33
6. Zaključak.....	35
7. Reference	37

1. Uvod

Razvoj mobilnih uređaja, GPS sistema, IoT senzora i sistema video nadzora doveo je do toga da se danas u realnom vremenu generišu ogromne količine podataka koji opisuju kretanje ljudi i vozila, kao i promene u njihovom okruženju. Ovi podaci su poznati kao prostorno-vremenski podaci. Kada se ovakvi podaci generišu kontinuirano, velikom brzinom i u velikom obimu, govorimo o velikim i brzim tokovima podataka (*Big Streaming Data*).

Tradicionalni pristupi analizi podataka, zasnovani na skladištenju podataka i njihovoj naknadnoj obradi, često nisu adekvatni u situacijama kada je neophodno donositi odluke ili generisati uvide u realnom vremenu, tako da su u ovakvim slučajevima neophodni sistemi koji mogu da obrađuju podatke u trenutku njihovog nastanka, sa minimalnom latencijom, uz mogućnost skaliranja u zavisnosti od intenziteta toka podataka. Kao odgovor na ove potrebe, razvijen je niz softverskih platformi, biblioteka i okvira namenjenih obradi tokova podataka (*stream processing*), među kojima se posebno izdvajaju Apache Flink, Apache Spark Structured Streaming i Kafka Streams. Ove platforme omogućavaju distribuiranu, skalabilnu i otpornu na greške obradu podataka u realnom vremenu, a u kombinaciji sa metodama mašinskog i dubokog učenja mogu se koristiti i za naprednu analitiku.

Cilj ovog rada je razvoj sistema za analizu velikih tokova prostorno-vremenskih podataka korišćenjem Apache Flink platforme. Sistem obrađuje podatke o kretanju vozila u realnom vremenu, detektuje događaje i generiše agregirane statistike koje opisuju ponašanje vozila u različitim geografskim zonama. Dodatno, vrši se analiza performansi sistema u zavisnosti od različitih konfiguracija, kako bi se ispitala skalabilnost i efikasnost implementiranog rešenja.

Struktura rada je organizovana na sledeći način. U drugom poglavlju se uvode osnovni teorijski pojmovi vezani za velike i brze tokove podataka, dok su u trećem opisane specifičnosti prostornih i prostorno-vremenskih podataka. Četvrto poglavlje sadrži pregled i komparativnu analizu relevantnih platformi, alata i biblioteka za obradu tokova podataka. U petom poglavlju opisuje se arhitektura sistema i implementacija, prikazani su vizuelni rezultati i izvršena evaluacija ponašanja sistema u zavisnosti od različitih konfiguracija. U šestom poglavlju se iznose zaključci rada, ograničenja sistema i mogući pravci za dalje unapređenje.

2. Big Data i Big Streaming Data

2.1. Big Data

Termin *Big Data* odnosi se na skupove podataka čiji obim, brzina generisanja i raznovrsnost prevazilaze mogućnosti tradicionalnih sistema za upravljanje bazama podataka i alata za analizu. Ovi skupovi podataka koji mogu biti strukturirani, polustrukturirani i nestrukturirani rastu, eksponencijalno po obimu i kompleksnosti tako da je za njihovu kolekciju, obradu i analizu potrebno koristiti nove tehnologije radi uspešnog donošenja informisanih odluka i rešavanja biznis problema u različitim oblastima.

Karakteristike *Big Data* sistema najčešće se opisuju kroz model poznat kao “3V”:

- *Volume* (obim) - Ovo je najčešće povezana karakteristika sa pojmom *Big Data*. Količina podataka koja se generiše najčešće se meri u terabajtima ili petabajtima.
- *Velocity* (brzina) - Odnosi se na tempo kojim se podaci generišu, u realnom vremenu ili približno realnom vremenu.
- *Variety* (raznovrsnost) - Podaci u savremenim sistemima su često različitih formata i strukture (strukturirani, polustrukturirani, nestrukturirani), a njihova heterogenost proizilazi i zbog različitosti izvora gde se ovi podaci generišu.

Dodatno se često koriste još 2 dimenzije tako da zajedno čine “5V” model:

- *Veracity* (pouzdanost podataka) - Zbog ogromne količine podataka često se među podacima mogu naći i netačni podaci. Sistem koji obrađuje velike skupove podataka treba da garantuje pouzdanost podataka, što se postiže mehanizmima validacije, filtriranja, čišćenja i transformacijama.
- *Value* (vrednost podataka) - Dobijanje vrednosti iz podataka je suština samog prikupljanja podataka. Vrednost podataka može da se ogleda u efikasnijem donošenju odluka i optimizaciji troškova.

2.1.1. Prednosti *Big Data*

Big data sistemi su transformisali način na koji organizacije prikupljaju uvide i donose strateške odluke. Studija časopisa *Harvard Business Review* pokazala je da su kompanije vođene podacima (*data-driven*) profitabilnije i inovativnije od svojih konkurenata. Organizacije koje efikasno koriste velike podatke i veštačku inteligenciju prijavile su da nadmašuju svoje konkurente u ključnim poslovnim metrikama, uključujući operativnu efikasnost, rast prihoda i korisničko iskustvo. Najznačajnije prednosti primene velikih tokova podataka:

- Poboljšano donošenje odluka - Analiza velikih skupova podataka omogućava organizacijama da otkriju obrasce i trendove koji vode do informisanih odluka.
- Unapređeno korisničko iskustvo – Veliki podaci omogućavaju kompanijama da razumeju ponašanje kupaca na mnogo detaljnijem nivou, otvarajući put za prilagođenije interakcije.

- Povećana operativna efikasnost - Podaci u realnom vremenu omogućavaju organizacijama da optimizuju poslovanje i smanje gubitke.
- Prilagodljiv razvoj proizvoda - Uvidi dobijeni analizom velikih podataka pomažu kompanijama da odgovore na potrebe korisnika i poboljšavaju svoje proizvode.
- Optimizovano određivanje cena - Veliki podaci omogućavaju organizacijama da usavrše strategije cena na osnovu tržišnih uslova u realnom vremenu.
- Unapređeno upravljanje rizicima i otkrivanje prevara - *Big data* omogućavaju organizacijama da proaktivno identifikuju i prate rizike [1].

2.1.2. Izazovi u radu sa Big Data sistemima

Iako veliki podaci nude ogroman potencijal, donose i značajne izazove, posebno u pogledu obima i brzine. Neki od najvećih izazova uključuju:

- Kvalitet i upravljanje podacima - Održavanje tačnosti može biti složen poduhvat, posebno sa masovnim količinama informacija koje neprestano pritiču sa društvenih mreža, IoT uređaja i drugih izvora.
- Skalabilnost - Kako podaci rastu, organizacije moraju da proširuju sisteme za skladištenje i obradu kako bi održale korak.
- Privatnost i bezbednost - Često postoje propisi koji zahtevaju stroge mere privatnosti i bezbednosti podataka, kao što su snažne kontrole pristupa i enkripcija kako bi se sprečio neovlašćeni pristup podacima, a ispunjavanje ovih zahteva može biti teško kada su skupovi podataka masovni i kada se neprestano menjaju.
- Složenost integracije - Kombinovanje različitih tipova podataka iz više izvora može biti tehnički zahtevno.
- Kvalifikovana radna snaga - Rad sa velikim podacima zahteva specijalizovane veštine, a mnoge organizacije se suočavaju sa stalnim izazovima u pronalaženju stručnjaka [1].

2.2. Big Streaming Data

Kod *Big Streaming Data* sistema naglasak je na dimenziji brzine. Podaci u ovim sistemima se generišu kontinuirano i moraju se obrađivati u realnom vremenu ili skoro u realnom vremenu. Obrada podrazumeva da se svaki novi podatak (ili mali skup podataka) obrađuje odmah po prijemu, čime se značajno smanjuje latencija između nastanka podatka i dobijanja korisnog rezultata. Ovakav pristup je neophodan u sistemima gde odluke moraju biti donesene u kratkom vremenskom roku npr. u sistemima za upravljanje saobraćajem, detekciju prevara, praćenje stanja industrijskih postrojenja ili analizu kretanja ljudi i vozila.

Podaci u ovim sistemima odlikuju se specifičnim osobinama koje ih bitno razlikuju od tradicionalnih podataka. Oni su pre svega neograničeni, što znači da pristižu u sistem neprekidno i bez vremenski definisanog kraja. Istovremeno su izuzetno heterogeni jer dolaze iz različitih izvora, noseći sa sobom različite formate i šeme. Karakteriše ih visoka brzina generisanja i obrade, kao i veliki obim koji nastaje akumulacijom neprekidnih tokova sitnih

događaja u velike skupove podataka. Svaki zapis u toku je jedinstven i po pravilu predstavlja događaj vezan za tačno određeni trenutak, zbog čega su ovi podaci izuzetno vremenski osetljivi, odnosno njihova vrednost rapidno opada kako stare. Na kraju, ovi tokovi su često nesavršeni i predstavljaju veliki inženjerski izazov, budući da redovno sadrže greške, praznine, nekonzistentnosti ili anomalije poput događaja koji pristižu van hronološkog redosleda [2].

Streaming sistemi se opisuju kroz nekoliko ključnih koncepata:

- Izvor podataka - generiše kontinuirani tok događaja,
- Obrada toka - uključuje operacije kao što su filtriranje, transformacija, agregacija i spajanje tokova,
- Vremenski prozori - omogućavaju da se agregacije i analize vrše nad ograničenim vremenskim ili količinskim segmentom toka,
- Garancija obrade - semantike *at-least-once*, *at-most-once* ili *exactly-once* definišu kako sistem rukuje gubitkom ili dupliranjem podataka u slučaju greške.

2.2.1. Prednosti Big Streaming Data sistema

Najznačajnije prednosti *streaming* sistema:

- Mala latencija i obrada u realnom vremenu - Rezultati obrade su dostupni maltene odmah po prijemu novih podataka, što je od presudnog značaja u primenama poput detekcije prevara u finansijskim transakcijama, upravljanja saobraćajem ili praćenja kretanja vozila, gde kašnjenje od nekoliko minuta može imati ozbiljne posledice.
- Kontinuirano praćenje i detekcija anomalija - Pošto se podaci analiziraju čim stignu, sistem može odmah da prepozna neobičan obrazac (npr. vozilo koje se kreće u pogrešnom smeru, nagli pad brzine saobraćaja na određenoj deonici) i da pokrene odgovarajuću reakciju bez čekanja na sledeći *batch* ciklus.
- Skalabilnost - Moderni *streaming* okviri projektovani su da podržavaju horizontalno skaliranje. Dodavanjem novih čvorova u klaster sistem može da podnese povećan intenzitet toka podataka bez prekida u radu, što je korisno u scenarijima sa neravnomernim opterećenjem (npr. jutarnje i popodnevne saobraćajne gužve).
- Inkrementalno učenje i ažuriranje modela - U kombinaciji sa metodama mašinskog učenja, *streaming* platforma može kontinuirano da ažurira model na osnovu novih podataka, čime se model prilagođava promenama u ponašanju sistema bez potrebe za ponovnim treniranjem na celokupnom istorijskom skupu podataka.
- Smanjena potreba za skupim dugoročnim skladištenjem podataka - Budući da se vrednost *streaming* podataka rapidno smanjuje kako stare, mnoge primene zahtevaju samo rezultate analize, a ne i sirove podatke, što znači da se sirovi tok može obraditi i odbaciti, uz čuvanje samo agregiranih uvida.

2.2.2. Izazovi u radu sa Big Streaming Data sistemima

Uprkos prednostima koje nude, sistemi za obradu velikih tokova podataka nose i skup specifičnih izazova koji ih čine znatno složenijim za projektovanje, implementaciju i održavanje u odnosu na klasične *batch* sisteme:

- Upravljanje stanjem - Operacije poput agregacija kroz vremenske prozore, praćenja sesija korisnika ili detekcije obrazaca koji se protežu kroz više uzastopnih događaja zahtevaju da sistem pamti određene informacije između uzastopnih događaja, odnosno da održava stanje. U distribuiranom okruženju, gde se tok podataka obrađuje paralelno na više čvorova, konzistentno i pouzdano upravljanje stanjem predstavlja ozbiljan izazov.
- Garancija obrade - U slučaju greške ili pada jednog od čvorova klastera, sistem mora da obezbedi da se podaci ne izgube i ne obrade dvaput. Implementacija *exactly-once* semantike izuzetno je složena u distribuiranom *streaming* okruženju i zahteva sofisticirane mehanizme za pamćenje stanja i oporavak.
- Problem kasnih i neurednih događaja - Podaci ne pristižu uvek onim redosledom kojim su nastali zbog mrežnih kašnjenja, privremenog gubitka veze ili kašnjenja na strani uređaja, što sve može uzrokovati da događaji stignu u sistem znatno nakon svog nastanka. Sistem mora da odluči koliko se čeka na kasne događaje pre nego što zatvori vremenski prozor i emituje rezultat, što uvek predstavlja kompromis između tačnosti rezultata i latencije sistema.
- Otpornost na greške i visoka dostupnost - Za razliku od *batch* sistema koji u slučaju greške mogu jednostavno ponovo pokrenuti obradu od početka, *streaming* sistemi moraju da obezbede kontinuiran rad čak i u slučaju pada pojedinih komponenti, uz minimalan gubitak podataka i minimalno kašnjenje u oporavku.
- Monitoring i debugovanje - U *streaming* sistemu, gde podaci kontinuirano teku i gde sistem mora reagovati u realnom vremenu, praćenje ispravnosti obrade, dijagnostika grešaka i testiranje ispravnosti logike obrade zahtevaju specijalizovane alate i pristupe koji su sami po sebi netrivialni.

3. Prostorni i prostorno-vremenski podaci

3.1. Prostorni podaci

Prostorni podaci (*spatial data*) su podaci koji, pored svoje vrednosti, sadrže i informaciju o lokaciji u prostoru. Lokacija je najčešće izražena kroz geografske koordinate (geografska širina i dužina), ali može biti označena i drugim prostornim reprezentacijama (tačka, linija i poligon). Prostorni podaci se javljaju u brojnim oblastima: geografskim informacionim sistemima (GIS), navigacionim sistemima, analizi urbanizma, praćenju vozila, mobilnim aplikacijama i mnogim drugim.

3.2. Prostorno-vremenski podaci

Kada se prostornoj dimenziji podataka doda i vremenska komponenta, odnosno kada se prati kako se prostorni podaci menjaju tokom vremena, dobijaju se prostorno-vremenski podaci (*spatio-temporal data*). Najčešći primer prostorno-vremenskih podataka su trajektorije kretanja koje predstavljaju niz uzastopnih parova lokacije i vremenskog trenutka, koji opisuju putanju kretanja nekog objekta (npr. vozila ili pešaka). Pored trajektorija, u ovu kategoriju spadaju i podaci koji opisuju promene stanja okruženja u prostoru i vremenu, kao što su promene gustine saobraćaja, meteorološki podaci, nivoi zagađenja vazduha ili promene u korišćenju zemljišta.

3.3. Obrada podataka tokova prostorno-vremenskih podataka

3.3.1 Batch processing

Batch processing podrazumeva da se podaci prikupljaju tokom određenog vremenskog perioda, skladište, a zatim obrađuju u celini, najčešće u unapred definisanim intervalima (npr. svakih sat vremena ili jednom dnevno). Ovaj pristup je pogodan za analize koje ne zahtevaju trenutne rezultate, kao što su generisanje izveštaja, treniranje modela mašinskog učenja na istorijskim podacima ili analize trendova na velikim, statičkim skupovima podataka. Prednost *batch* obrade je u tome što omogućava obradu velikih količina podataka uz visoku propusnost, ali po cenu veće latencije. Vreme između nastanka podatka i dobijanja rezultata može biti značajno.

3.3.2. Stream processing

Stream processing podrazumeva obradu podataka odmah po njihovom nastanku, bez potrebe da se prethodno skladište u celosti. Ovaj pristup omogućava značajno manju latenciju i pogodan je za primene koje zahtevaju brzu reakciju na nove podatke npr, detekciju anomalija

u kretanju vozila u realnom vremenu, generisanje upozorenja o saobraćajnim gužvama ili praćenje gustine kretanja ljudi na javnim mestima. *Streaming* sistemi često koriste koncept neograničenih tokova podataka, gde se podaci obrađuju kroz vremenske prozore, a rezultati se ažuriraju inkrementalno, kako novi podaci stižu.

Budući da su tokovi podataka po svojoj prirodi neograničeni i kontinuirani, nije moguće izvršiti agregacije ili analize nad celim tokom odjednom. Tok se logički deli na konačne segmente odnosno vremenske prozore, nad kojima se zatim primenjuju operacije poput brojanja, sumiranja, računanja proseka ili složenijih analitičkih funkcija. Izbor odgovarajućeg tipa prozora direktno utiče na tačnost i relevantnost rezultata analize. Tipovi prozora koji postoje:

- *Tumbling windows* - Dele tok podataka na segmente fiksne, jednake dužine koji se ne preklapaju. Svaki događaj pripada tačno jednom prozoru. Ovaj tip prozora pogodan je za periodične agregacije, kao što je računanje prosečne brzine kretanja vozila u svakom minutu ili broj pešaka koji su prošli kroz određenu zonu u svakom petominutnom intervalu.
- *Sliding windows* - Imaju fiksnu dužinu, ali se pomeraju za korak koji je manji od same dužine prozora, što znači da se uzastopni prozori međusobno preklapaju. Isti događaj može biti uključen u više uzastopnih prozora, što omogućava praćenje promena kroz vreme sa finijom vremenskom rezolucijom. Klizni prozori su pogodni za detekciju trendova i anomalija, na primer za praćenje da li se gustina saobraćaja u nekoj zoni povećava tokom poslednjih pet minuta.
- *Session windows* - Ne koriste fiksnu dužinu, već se dinamički formiraju na osnovu aktivnosti samog toka podataka. Prozor ostaje otvoren sve dok postoji aktivnost, a zatvara se tek kada protekne unapred definisani period neaktivnosti. Ovaj tip prozora posebno je pogodan za analizu kretanja gde su periodi aktivnosti prirodno razdvojeni pauzama. Na primer, analiza putovanja jednog vozila, gde se svako putovanje tretira kao zasebna sesija odvojena periodom mirovanja.

Pored izbora tipa prozora, u *streaming* sistemima je neophodno jasno razlikovati dva tipa vremena:

- Vreme događaja (*event time*) - Predstavlja trenutak kada je događaj nastao na izvoru.
- Vreme obrade (*processing time*) - Predstavlja trenutak kada je događaj stigao u sistem za obradu, što može biti znatno kasnije zbog kašnjenja u mreži ili privremenog gubitka veze uređaja.

Za tačnu analizu prostorno-vremenskih podataka neophodno je raditi u *event time* domenu, jer jedino on odražava stvarni redosled i trenutke nastanka događaja. Međutim, to uvodi dodatnu složenost jer sistem ne može unapred da zna kada su stigli svi događaji koji pripadaju nekom vremenskom prozoru, jer kasni događaji mogu pristizati i dugo nakon nominalnog kraja prozora. Ovaj problem rešava se mehanizmom vodenog žiga (*watermark*), koji predstavlja pretpostavku sistema o tome do koje tačke u *event time*-u su svi podaci stigli. Kada *watermark* pređe granicu nekog vremenskog prozora, sistem smatra da su svi relevantni događaji stigli i zatvara prozor, emitujući konačan rezultat. *Watermark* se obično definiše kao maksimalno viđeno vreme događaja u sistemu umanjeno za određenu toleranciju kašnjenja, čime se postiže kompromis između tačnosti rezultata i latencije sistema.

3.3.3. Poređenje batch i stream processing-a u obradi prostorno-vremenskih podataka

Iako obe metode upravljaju velikim količinama podataka, one služe različitim poslovnim potrebama i zahtevaju potpuno drugačije arhitekture. Ključne razlike se ogledaju u samom modelu obrade, infrastrukturnim potrebama i zahtevima za performansama.

Dok model obrade kod *batch* procesiranja objedinjuje i analizira skupove podataka u paketima u tačno određenim vremenskim intervalima, obrada podataka u toku koristi alate za obradu u realnom vremenu kako bi analizirala podatke čim oni pristignu. To omogućava *streaming* sistemima da trenutno generišu uvide i pokrenu akcije, dok *batch* sistemi rade po periodičnom rasporedu.

Kada je reč o infrastrukturi, *batch* sistemi često koriste tradicionalna skladišta podataka i alate za analitiku kao što su *data warehouse-i*, dok *streaming* sistemi zahtevaju specijalizovane okvire i platforme koje su namenski napravljene za upravljanje tokovima podataka u realnom vremenu.

U pogledu performansi, *batch* sistemi mogu optimizovati korišćenje resursa tokom planiranih pokretanja, dok obrada tokova podataka zahteva sisteme otporne na greške koji obezbeđuju nisko kašnjenje, što znači da *streaming* sistemi moraju procesirati podatke u realnom vremenu i bez odlaganja, čak i u uslovima visokog opterećenja ili nepredviđenih problema.

Organizacije obično biraju između *batch* i *streaming* obrade na osnovu obima podataka, potreba za malim kašnjenjem i poslovnih ciljeva, pri čemu mnoge od njih kombinuju oba pristupa unutar jedinstvene mreže podataka kako bi uspešno upravljale različitim vrstama zadataka [2].

Kriterijum	Batch processing	Stream processing
Latencija	Visoka (minuti ili sati)	Niska (milisekunde ili sekunde)
Propusnost	Visoka	Srednja - visoka
Tačnost prostornih operacija	Visoka	Ograničena prozorom
Upravljanje stanjem	Jednostavno	Složeno
Tolerancija na kasne događaje	Potpuna	Ograničena
Složenost implementacije	Niža	Viša

Primena	Istorijska analiza, treniranje modela	Praćenje u realnom vremenu, detekcija anomalija
---------	---------------------------------------	---

Tabela 1. Poređenje *batch* i *stream processing-a*

3.3.4. Primena stream procesinga na prostorno-vremenske podatke

Geofencing u realnom vremenu jedna je od najčešćih primena *stream* obrade na prostorno-vremenske podatke. *Geofencing* podrazumeva detekciju trenutka kada neki objekat (vozilo, pešak ili mobilni uređaj) ulazi u unapred definisanu geografsku zonu ili izlazi iz nje. U *streaming* kontekstu, svaki novi GPS zapis se upoređuje sa skupom definisanih zona, a odgovarajući događaj (ulazak ili izlazak) generiše se odmah po detekciji.

Rekonstrukcija i analiza trajektorija u realnom vremenu podrazumeva kontinuirano nadovezivanje uzastopnih GPS zapisa istog objekta u koherentnu putanju kretanja. Ovaj zadatak nije trivijalan jer GPS zapisi mogu stizati van redosleda, mogu sadržati greške merenja ili praznine usled gubitka signala, a sistem mora da vodi stanje za svaki praćeni objekat posebno. Na osnovu rekonstruisane trajektorije mogu se u realnom vremenu izračunavati parametri kretanja poput trenutne brzine, ubrzanja, smera kretanja i odstupanja od očekivane rute.

Prostorna agregacija kroz vremenske prozore kombinuje koncepte vremenskih prozora sa prostornim operacijama. Na primer, korišćenjem kliznog prozora od pet minuta može se kontinuirano pratiti gustina vozila u svakoj ćeliji prostorne mreže, detektovati formiranje saobraćajnih gužvi ili pratiti promene u gustini kretanja pešaka na javnim mestima.

Detekcija prostorno-vremenskih anomalija još je jedna važna primena. *Streaming* sistem može u realnom vremenu da identifikuje neobične obrasce kretanja, npr. vozilo koje se kreće neuobičajeno brzo ili sporo za datu lokaciju i vreme, objekat koji se ponaša drugačije od istorijskog obrasca ili grupu objekata čije se kretanje naglo menja. Ovakve analize često kombinuju *streaming* obradu sa unapred istreniranim modelima mašinskog učenja koji se primenjuju na svaki novi događaj ili agregat u realnom vremenu.

Prostorno spajanje tokova odnosi se na situacije kada je potrebno kombinovati dva ili više istovremenih tokova prostornih podataka, npr. tok GPS lokacija vozila sa tokom podataka o meteorološkim uslovima ili tokom podataka o saobraćajnim incidentima. Rezultat spajanja je obogaćen tok koji za svako vozilo sadrži i informaciju o uslovima u njegovom neposrednom okruženju, što omogućava složenije analize poput predikcije kašnjenja u zavisnosti od vremenskih uslova.

3.4. Izazovi obrade prostorno-vremenskih tokova podataka

Obrada prostorno-vremenskih podataka u realnom vremenu nameće skup specifičnih izazova koji ne postoje ili su znatno manje izraženi u klasičnoj *batch* obradi statičkih skupova podataka. Ovi izazovi proizilaze iz kombinacije dve dimenzije složenosti, prostorne i vremenske, u kontekstu kontinuiranih, brzih i često nesavršenih tokova podataka.

Pre svega, potrebno je definisati odgovarajuće prostorne operacije, poput proračuna udaljenosti između dve tačke, provere da li se tačka nalazi unutar određenog poligona, prostornog klasterovanja ili detekcije prostornih obrazaca. Računska zahtevnost ovih operacija je skupa. U kontekstu *batch* obrade, ova zahtevnost se može ublažiti paralelizacijom i prethodnim indeksiranjem podataka. U *streaming* kontekstu prostorne operacije moraju biti izvršene u realnom vremenu, nad podacima koji kontinuirano pristižu, što zahteva efikasne prostorne indeksne strukture i pažljivo projektovane algoritme koji mogu da rade inkrementalno, bez ponovnog procesiranja celokupnog skupa podataka pri svakom novom događaju.

U idealnom scenariju, podaci bi pristizali u sistem tačno onim redosledom kojim su nastali. U praksi se međutim dešava da GPS uređaji, mobilni telefoni i drugi senzori koji generišu prostorno-vremenske podatke često šalju podatke sa zakašnjenjem, usled lošeg signala, privremenog gubitka veze ili kašnjenja u mreži. Posledica toga je da događaji pristižu van hronološkog redosleda, što otežava tačnu rekonstrukciju trajektorije kretanja i ispravnu agregaciju unutar vremenskih prozora. *Streaming* sistemi moraju da implementiraju mehanizme za rukovanje ovakvim situacijama, poput koncepta vodenog žiga, koji definiše do kog stepena kašnjenja sistem još uvek prihvata i uzima u obzir kasno pristigle događaje pre nego što zatvori vremenski prozor i emituje rezultat.

Analize bazirane na *processing time*-u mogu dati pogrešnu sliku o stvarnom kretanju objekta, pa je za tačnu analizu trajektorija i prostorno-vremenskih obrazaca neophodno raditi u *event time* domenu, uz prihvatanje dodatne složenosti u upravljanju vremenskim prozorima.

Klasične prostorne indeksne strukture projektovane su za statičke skupove podataka gde se jednom dodeljen indeks koristi za niz upita. U *streaming* kontekstu sa novim podacima koji neprestano pristižu prostorni indeks mora biti ažuriran inkrementalno, bez narušavanja performansi tekuće obrade. Ovo je posebno izazovno u scenarijima gde se prati veliki broj objekata čije se lokacije ažuriraju svakih nekoliko sekundi.

Tokovi prostorno-vremenskih podataka mogu biti izuzetno intenzivni, npr. sistem koji prati kretanje svih vozila u velikom gradu može generisati milione GPS zapisa u minuti. Sistemi za obradu moraju biti sposobni da horizontalno skaliraju, odnosno da dodavanjem novih čvorova u distribuirani klaster povećaju kapacitet obrade, bez prekida u radu. Istovremeno, prostorne operacije koje se izvode nad ovako velikim tokovima moraju biti pažljivo optimizovane kako bi se izbeglo prekomerno korišćenje memorije i procesorskih resursa.

4. Tehnologije za obradu tokova prostorno-vremenskih podataka

U praksi, mnogi moderni sistemi za obradu podataka podržavaju oba pristupa *batch* i *stream* obradu, a neki idu i korak dalje implementirajući tzv. unifikovani model obrade.

Izbor između *batch* i *stream* pristupa, ili njihove kombinacije, zavisi od konkretnih zahteva primene, pre svega od toga koliko je kritično da rezultati analize budu dostupni u realnom vremenu, kao i od prirode i intenziteta toka podataka koji se obrađuje. U kontekstu analize kretanja ljudi i vozila, gde su pravovremene informacije od ključnog značaja, *streaming* pristup se nameće kao prirodan izbor, što ovaj rad i opredeljuje ka daljem proučavanju tehnologija namenjenih ovoj vrsti obrade.

4.1. Apache Kafka

Apache Kafka je distribuirana platforma za *streaming* podataka otvorenog koda, prvobitno razvijena u kompaniji LinkedIn, a od 2011. godine dostupna kao projekat Apache Software Foundation. Kafka nije sistem za obradu podataka u klasičnom smislu, već je primarna uloga pouzdano prikupljanje, čuvanje i prosleđivanje velikih tokova događaja između proizvođača (*producer*) i potrošača (*consumer*) podataka, zbog čega se često opisuje kao distribuirani sistem za razmenu poruka (*message broker*) ili platforma za upravljanje tokovima događaja (*event streaming platform*).

Osnovna apstrakcija je *topic*, koji predstavlja imenovani kanal u koji proizvođači upisuju poruke, a iz kojeg potrošači čitaju. Svaki *topic* je podeljen na jednu ili više particija, koje su distribuirane po čvorovima klastera (*broker-ima*). Ovakva organizacija omogućava horizontalno skaliranje, povećanjem broja particija povećava se i kapacitet sistema za paralelnu obradu. Svaka poruka unutar particije dobija jedinstveni redni broj (*offset*) koji potrošačima omogućava da čitaju poruke tačno određenim redosledom i da u slučaju greške nastave čitanje od tačke na kojoj su stali [3].

Kafka garantuje visoku propusnost i nisku latenciju zahvaljujući nekoliko arhitekturnih odluka: poruke se upisuju sekvencijalno na disk (što je brže od nasumičnog pristupa), replikacija podataka između *broker-a* obezbeđuje toleranciju na greške, a odvojenost proizvođača i potrošača omogućava nezavisno skaliranje obe strane sistema. Kafka čuva poruke na disku tokom konfigurisanog vremenskog perioda, što znači da potrošači mogu čitati podatke i retroaktivno. Ova osobina je posebno korisna za ponovno čitanje istorijskih tokova podataka prilikom procesa testiranja [3].

U kontekstu prostorno-vremenskih podataka, Kafka se najčešće koristi kao ulazni sloj sistema tako što prikuplja GPS zapise iz vozila, mobilnih uređaja ili senzora i prosleđuje ih sistemu za obradu poput Apache Flink-a ili Spark-a. Tipičan scenario podrazumeva da svako vozilo ili senzor upisuje svoje GPS zapise u odgovarajući Kafka *topic*, odakle ih *streaming* sistem preuzima, obrađuje i upisuje rezultate u izlazni *topic* ili bazu podataka. Kafka na ovaj način

služi kao sloj između izvora podataka i sistema za obradu, kontrolišući nagle skokove u intenzitetu toka i garantujući da se nijedan zapis neće izgubiti čak i ako sistem za obradu privremeno nije dostupan.

4.2. Apache Spark Streaming

Apache Spark je distribuirani okvir za obradu podataka otvorenog koda. Originalno je projektovan kao sistem za *batch* obradu koji podatke obrađuje u *in-memory*, čime postiže značajno veće brzine u odnosu na prethodne sisteme zasnovane na MapReduce paradigmi. Podrška za *streaming* obradu dodata je kroz komponentu Spark Streaming, a kasnije je zamenjena modernijim pristupom Structured Streaming.

Structured Streaming tretira tok podataka kao neograničenu tabelu koja se kontinuirano popunjava novim redovima kako novi podaci pristižu. Korisnik definiše transformacije nad ovom tabelom koristeći isti SQL ili DataFrame API koji se koristi i za *batch* obradu, a Spark interno brine o inkrementalnom izvršavanju tih transformacija na svakom novom *micro-batch-u* podataka. Ovaj pristup značajno pojednostavljuje razvoj *streaming* aplikacija jer ne mora da se razmišlja o razlici između *batch* i *streaming* konteksta jer isti kod funkcioniše u oba slučaja [4].

Spark Structured Streaming podržava obradu u *event time* domenu, upravljanje vremenskim prozorima sva tri tipa i mehanizam *watermark-a* za rukovanje kasnim događajima. *Exactly-once* garancija obrade obezbeđuje se kroz mehanizam *checkpointing-a* koji periodično snima stanje obrade na trajni medijum, omogućavajući oporavak bez gubitka podataka u slučaju greške [4].

Za obradu prostorno-vremenskih podataka, Spark se može proširiti bibliotekom Apache Sedona (ranije poznata kao GeoSpark), koja dodaje podršku za prostorne tipove podataka i prostorne operacije direktno nad Spark DataFrame strukturama. Sedona omogućava efikasno izvođenje operacija poput prostornog spajanja, *geofencinga* i prostornih agregacija nad velikim tokovima prostornih podataka, koristeći prostorne indeksne strukture za ubrzanje upita.

Spark se odlikuje bogatim ekosistemom biblioteka za mašinsko i duboko učenje: MLlib za klasične ML algoritme i integracijama sa TensorFlow i PyTorch za duboko učenje. To ga čini pogodnim za primene koje kombinuju *streaming* obradu sa naprednom analitikom i predikcijom.

4.3. Apache Flink

Apache Flink je distribuirani okvir za obradu tokova podataka otvorenog koda i deo je projekta Apache Software Foundation. Za razliku od Spark-a, koji je originalno projektovan kao *batch* sistem sa naknadno dodatom podrškom za *streaming*, Flink je od samog početka projektovan

kao *native streaming* sistem u kome je *streaming* osnovna paradigma obrade, a *batch* obrada tretira se kao specijalan slučaj *streaming* obrade sa ograničenim tokom podataka.

Flink obrađuje svaki događaj pojedinačno čim stigne, bez grupisanja u mikro-pakete kao što to čini Spark. Ovakav pristup je poznat kao *true streaming* ili *event-at-a-time* obrada što rezultuje znatno manjom latencijom u odnosu na *micro-batch* sisteme, što ga čini posebno pogodnim za primene sa strogim zahtevima u pogledu vremena odziva [5].

Flink nudi naprednu podršku za upravljanje vremenskim prozorima i *event time* obradu. Mehanizam *watermark-a* u Flink-u je izuzetno fleksibilan i omogućava precizno podešavanje tolerancije kašnjenja za različite tokove podataka. Pored standardnih tipova prozora, Flink podržava i *custom window assigner* mehanizam koji omogućava definisanje proizvoljnih pravila za dodeljivanje događaja prozorima, što je korisno u složenim scenarijima analize trajektorija [5].

Upravljanje stanjem je jedna od najsnažnijih karakteristika Flink-a. Flink podržava *keyed state* koje predstavlja stanje koje se čuva posebno za svaki ključ u toku podataka. Ovo je direktno primenjivo na praćenje kretanja gde svaki praćeni objekat predstavlja jedan ključ čije se stanje čuva i ažurira kontinuirano. Stanje se periodično snima kroz mehanizam *distributed snapshots* (zasnovan na Chandy-Lamport algoritmu), koji garantuje *exactly-once* semantiku obrade uz minimalan uticaj na performanse [5].

Za prostorno-vremensku analizu, Flink se može kombinovati sa bibliotekom GeoFlink, koja proširuje Flink API-jem za prostorne operacije i prostorno-vremenske upite nad *streaming* podacima. GeoFlink podržava prostorne tipove podataka, prostorno indeksiranje kroz *geohash* i prostorne prozorske upite koji kombinuju prostornu i vremensku dimenziju filtriranja.

4.4. Kafka Streams

Kafka Streams je biblioteka za obradu tokova podataka koja je deo Apache Kafka ekosistema. Za razliku od Flink-a i Spark-a, koji su samostalne distribuirane platforme koje zahtevaju poseban klaster za pokretanje, Kafka Streams je laka Java biblioteka koja se ugrađuje direktno u aplikaciju i izvršava se unutar nje, bez potrebe za posebnom infrastrukturom za obradu. Jedina infrastrukturna zavisnost je postojeći Kafka klaster koji služi kao izvor i odredište podataka, kao i za koordinaciju i upravljanje stanjem.

Kafka Streams koristi apstrakciju KStream za predstavljanje neograničenih tokova događaja i KTable za predstavljanje tabela koje se ažuriraju na osnovu toka, što odgovara konceptu *materialized view-a* nad tokom podataka. Kombinovanje KStream i KTable apstrakcija omogućava elegantno modelovanje složenih *streaming* scenarija, poput spajanja toka GPS zapisa sa tabelom koja sadrži trenutne metapodatke o vozilima [6].

Kafka Streams podržava upravljanje lokalnim stanjem kroz ugrađene *state store-ove*, koji se

interno replikuju nazad u Kafka *topic*-e čime se obezbeđuje tolerancija na greške. Podrška za vremenske prozore i *event time* obradu postoji, ali je manje sofisticirana u poređenju sa Flink-om ili Spark-om, što Kafka Streams čini manje pogodnim za složene prostorno-vremenske analize sa strogim zahtevima u pogledu tačnosti upravljanja vremenom [6].

Glavna prednost Kafka Streams leži u jednostavnosti operativnog upravljanja. Skaliranje se postiže prostim pokretanjem više instanci iste aplikacije i to bez upravljanja posebnim klasterom za obradu. Ovo ga čini atraktivnim izborom za mikroservisne arhitekture i primene srednje složenosti, gde bi kompleksnost upravljanja Flink ili Spark klasterom bila nepotrebna. U kontekstu prostorno-vremenskih podataka, Kafka Streams je pogodan za jednostavnije scenarije poput filtriranja i transformacije GPS tokova, ali za složenije analize poput rekonstrukcije trajektorija ili naprednog *geofencing*-a preporučuje se upotreba Flink-a ili Spark-a.

4.5. Poređenje tehnologija

Sve četiri opisane tehnologije adresiraju problem obrade tokova podataka, ali sa različitim pristupima, kompromisima i oblastima primene. Tabela 2 daje pregled ključnih karakteristika po kojima se ove tehnologije međusobno razlikuju.

Iz poređenja se može zaključiti da u praksi ove tehnologije nisu konkurenti već komplementarne komponente istog sistema. Tipična arhitektura sistema za obradu velikih tokova prostorno-vremenskih podataka koristi Kafku kao ulazni sloj za prikupljanje i transport GPS zapisa i podataka sa senzora, Flink ili Spark kao centralni sloj za obradu, prostornu analizu i primenu ML modela i odgovarajuće baze podataka ili vizualizacione alate kao izlazni sloj. Kafka Streams može biti dobar izbor za jednostavnije korake predobrade koji se mogu implementirati kao deo iste aplikacije koja konzumira Kafka *topic*-e, bez potrebe za posebnim klasterom za obradu.

Između Flink-a i Spark-a, izbor zavisi od konkretnih zahteva. Flink je bolji izbor kada je ključna minimalna latencija i napredna *event time* semantika, što je tipično za praćenje kretanja u realnom vremenu sa strogim zahtevima u pogledu tačnosti. Spark je pogodniji kada je prioritet bogat ekosistem ML biblioteka i ujednačen API između *batch* i *streaming* koda, što je tipično za sisteme koji kombinuju istorijsku analizu trajektorija sa *streaming* predikcijom.

Kriterijum	Apache Kafka	Spark Structured Streaming	Apache Flink	Kafka Streams
Primarna uloga	Prikupljanje i transport podataka	Batch i stream obrada	Native stream obrada	Lagana stream obrada
Model obrade	Message broker	Mikro-batch	Event-at-a-time	Event-at-a-time
Latencija	Milisekunde	Sekunde	Milisekunde	Milisekunde
Upravljanje stanjem	Ograničeno	Srednje	Napredno	Srednje
Event time podrška	Osnovna	Dobra	Odlična	Osnovna
Exactly-once garancija	Da (uz konfiguraciju)	Da	Da	Da
Podrška za prostorne operacije	Ne	Da (Apache Sedona)	Da (GeoFlink)	Ne
ML/DL integracija	Ne	Odlična (MLlib, TensorFlow)	Dobra	Ograničena
Operativna složenost	Srednja	Visoka	Visoka	Niska
Pogodnost za prostorno-vremenske tokove	Kao ulazni sloj	Visoka	Visoka	Ograničena

Tabela 2. Poređenje karakteristika tehnologija za obradu tokova prostorno-vremenskih podataka

5. Projektovanje sistema za obradu prostorno-vremenskih tokova podataka

5.1. Opis problema

Cilj praktičnog dela ovog rada je razvoj sistema koji u realnom vremenu prikuplja, obrađuje i analizira prostorno-vremenske podatke o kretanju vozila javnog gradskog prevoza, primenjujući koncepte i tehnologije opisane u teorijskom delu rada. Kao izvor podataka korišćen je javno dostupan tok podataka o pozicijama autobusa sistema MTA (Metropolitan Transportation Authority) u Njujorku, koji u realnom vremenu objavljuje GPS pozicije, identifikatore vozila i linija, kao i operativni status svakog vozila u sistemu.

Na osnovu ovog toka podataka, sistem treba da izvrši nekoliko vrsta prostorno-vremenske analize: detekciju ulaska, izlaska i zadržavanja vozila u unapred definisanim geografskim zonama (*geofencing*), detekciju predugog zadržavanja vozila u zoni, praćenje sekvenci kretanja vozila između zona, kao i agregaciju broja vozila po zoni kroz vremenske prozore koja predstavlja gustinu saobraćaja. Sistem treba da bude projektovan kao tipičan *streaming pipeline* sa jasno razdvojenim slojem za prikupljanje podataka, slojem za obradu i analizu, slojem za skladištenje rezultata i slojem za vizualizaciju.

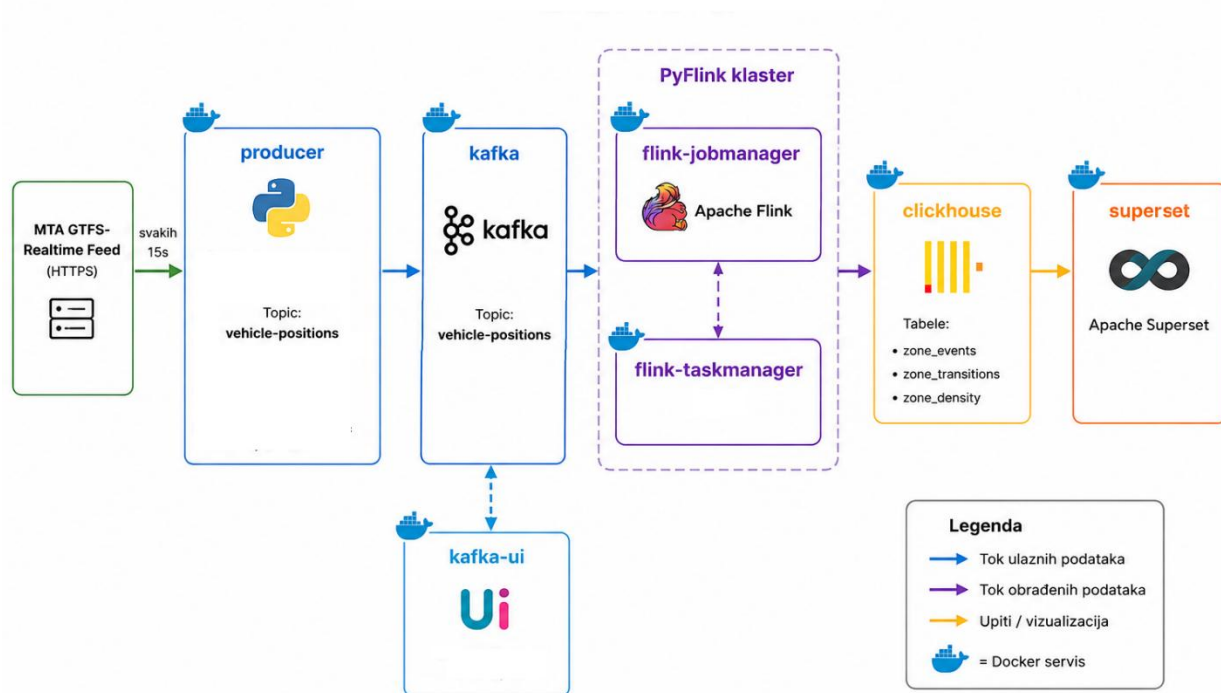
Za potrebe analize definisano je šest geografskih zona u Njujorku, izabranih tako da predstavljaju raznovrsne tipove lokacija: turističku/poslovnu zonu visoke gustine kretanja (Times Square), dva aerodroma (JFK i LaGuardia), glavnu železničku stanicu (Grand Central), poslovnu i stambenu četvrt u Bruklinu (Downtown Brooklyn i Williamsburg). Odabrani geografski centri i radijusi zona se mogu videti u tabeli 3.

Zona	Centar(lat,lon)	Radijus(m)
<i>Times Square</i>	40.758, -73.9855	400
<i>JFK Airport</i>	40.6413, -73.7781	1500
<i>Grand Central</i>	40.7527, -73.9772	400
<i>Downtown Brooklyn</i>	40.6928, -73.9903	600
<i>LaGuardia Airport</i>	40.7769, -73.8740	1200
<i>Williamsburg</i>	40.7081, -73.9571	500

Tabela 3. Pregled zona

5.2. Arhitektura sistema

Arhitektura razvijenog sistema sastoji se od šest komponenti, organizovanih u sistem kontejnerizovanih servisa. Slika 1 prikazuje opštu strukturu sistema, a u nastavku je opisana uloga svake komponente.



Slika 1. Dijagram arhitekture sistema

5.2.1. Izvor podataka - MTA Bus Time GTFS-Realtime feed

Polazna tačka sistema je javni GTFS-Realtime (GTFS-RT) feed kompanije MTA, koji predstavlja konkretnu implementaciju GTFS (General Transit Feed Specification) standarda. Dostupan je na adresi <https://gtfsrt.prod.obanyc.com/vehiclePositions>. To je standard otvorenog formata za razmenu podataka o javnom prevozu, koji su zajednički definisali Google i transportne agencije širom sveta. GTFS standard razdvaja podatke na statički deo GTFS-Static (raspored, rute, stanice, ažurira se periodično) i dinamički deo GTFS-Realtime (trenutne pozicije vozila, kašnjenja, upozorenja, ažurira se kontinuirano). U ovom radu korišćen je GTFS-Realtime Vehicle Positions feed, koji svakih ~10 sekundi objavljuje trenutno stanje svih aktivnih vozila u sistemu, enkodirano u binarnom Protocol Buffers (Protobuf) formatu.

5.2.2. Producer servis

Producer je Python servis koji periodično (na svakih 15 sekundi) šalje HTTP zahtev ka GTFS-

RT *feed*-u, parsira dobijeni Protobuf odgovor i iz njega izdvaja entitete tipa vozila. GTFS standard ovde definiše tačnu strukturu podataka koju producer očekuje i parsira, polja kao što su pozicija definisana geografskim koordinatama, identifikator vozila, identifikator linije i identifikator putovanja, čija je semantika detaljnije objašnjena u poglavlju 5.3. Svaki izdvojeni zapis se serijalizuje u JSON format i upisuje u Apache Kafka *topic*, pri čemu se kao ključ poruke koristi identifikator vozila, čime se obezbeđuje da sve poruke istog vozila završe na istoj Kafka particiji.

5.2.3. Apache Kafka

Kafka služi kao ulazni sloj sistema koji prihvata kontinuirani tok poruka od *producer* servisa i čini ih dostupnim za dalju obradu, nezavisno od trenutne dostupnosti ili brzine obrade *downstream* komponenti.

5.2.4. Apache Flink

Na osnovu poređenja prikazanog u poglavlju 4.5, za implementaciju sistema odabran je Apache Flink zbog podrške za *stream processing*, naprednog upravljanja stanjem, fleksibilne *event-time* semantike i male latencije obrade.

Flink predstavlja centralni sloj za obradu i analizu podataka. U sistemu su implementirana dva nezavisna Flink *job*-a, koji demonstriraju dva različita pristupa *stream processing*-u. Prvi, *geofencing job*, koristi stateful *KeyedProcessFunction* pristup za detekciju događaja po pojedinačnom vozilu, dok drugi, *density job*, koristi Flink-ov ugrađeni *window API* za agregaciju nad grupama vozila u vremenskim prozorima. Oba čitaju iz istog Kafka *topic*-a, obavljaju *geofencing* proveru korišćenjem koordinata svakog zapisa i upisuju rezultate u ClickHouse.

5.2.5. ClickHouse

ClickHouse baza podataka je korišćena kao skladište rezultata analize. Izbor ClickHouse-a, umesto klasične relacione baze, napravljen je zbog optimizovanog rada sa analitičkim upitima nad velikim brojem redova, kao i zbog ugrađene podrške za automatsko brisanje podataka starijih od definisanog perioda, što je relevantno za rad sa podacima koji vremenom gube na značaju, kao što je opisano u teorijskom delu rada u poglavlju 2.2.

5.2.6. Apache Superset

Apache Superset je korišćen kao sloj za vizualizaciju rezultata, povezan direktno na ClickHouse. Kroz Superset su konstruisani grafici koji prikazuju rezultate analiza, opisani su detaljnije u poglavlju 5.5.

5.3. Model i struktura podataka

GTFS standard organizuje podatke o javnom prevozu hijerarhijski, kroz tri ključna pojma: rutu, putovanje i stanicu. Ruta koja je identifikovana poljem *route_id*, predstavlja liniju javnog prevoza (npr. linija "M15") i ona se ne menja se iz dana u dan. Putovanje, identifikovano poljem *trip_id*, predstavlja jednu konkretnu vožnju te linije u određenom danu. Isto fizičko vozilo tokom dana ostvaruje više putovanja, svako sa svojim *trip_id* i unapred definisanim, fiksnim redosledom stanica. Stanica je identifikovana poljem *stop_id* i predstavlja fiksnu geografsku tačku na ruti.

Tabela 4 prikazuje strukturu JSON poruke koju *producer* upisuje u Kafka *topic vehicle-positions*.

Polje	Tip	Opis
<i>vehicle_id</i>	<i>string</i>	Jedinstveni identifikator vozila
<i>route_id</i>	<i>string</i>	Identifikator linije
<i>trip_id</i>	<i>string</i>	Identifikator konkretnog putovanja
<i>latitude</i>	<i>float</i>	Geografska širina
<i>longitude</i>	<i>float</i>	Geografska dužina
<i>bearing</i>	<i>float</i>	Smer kretanja vozila
<i>current_status</i>	<i>string</i>	Pozicija u odnosu na sledeću stanicu
<i>stop_id</i>	<i>string</i>	Identifikator sledeće/trenutne stanice
<i>timestamp</i>	<i>int</i>	Vreme nastanka zapisa
<i>ingested_at</i>	<i>string</i>	Vreme kada je <i>producer</i> preuzeo zapis

Tabela 4. Struktura poruke sa *topic*-a *vehicle-positions*

Rezultati analiza, dobijeni obradom ovih poruka, upisuju se u tri ClickHouse tabele:

- *zone_events* - sadrži ENTER, EXIT, DWELL i ALERT događaje za zone,
- *zone_transitions* - sadrži tranzicije vozila između parova zona,
- *zone_density* - sadrži gustinu vozila po zoni.

Struktura ovih tabela detaljno je opisana u nastavku u okviru tabela 5, 6 i 7.

Polje	Tip	Opis
<i>event_time</i>	DateTime64	Vreme nastanka zapisa
<i>processing_time</i>	DateTime64	Vreme procesuiranja zapisa
<i>vehicle_id</i>	String	Identifikator vozila
<i>route_id</i>	String	Identifikator rute
<i>zone_id</i>	String	Identifikator zone
<i>zone_name</i>	String	Ime zone
<i>event_type</i>	Enum8	Tip događaja
<i>latitude</i>	Float64	Geografska širina
<i>longitude</i>	Float64	Geografska dužina
<i>dwel_seconds</i>	UInt32	Vreme zadržavanja u zoni

Tabela 5. Struktura tabele *zone_events*

Polje	Tip	Opis
<i>transition_time</i>	DateTime64	Vreme prelaska u zonu identifikovanu sa <i>to_zone_id</i>
<i>vehicle_id</i>	String	Identifikator vozila
<i>route_id</i>	String	Identifikator rute
<i>from_zone_id</i>	String	Identifikator zone iz koje je vozilo došlo
<i>from_zone_name</i>	String	Ime zone iz koje je vozilo došlo
<i>to_zone_id</i>	String	Identifikator zone u koju vozilo prelazi
<i>to_zone_name</i>	String	Ime zone u koju vozilo prelazi
<i>travel_time_seconds</i>	UInt32	Trajanje prelaska

Tabela 6. Struktura tabele *zone_transitions*

Polje	Tip	Opis
<i>window_end</i>	DateTime64	Vreme završetka petominutnog vremenskog prozora za koji je izračunata agregacija
<i>route_ids</i>	Array(String)	Niz identifikatora linija čija su vozila bila prisutna u zoni tokom posmatranog prozora
<i>zone_id</i>	String	Identifikator zone
<i>zone_name</i>	String	Ime zone
<i>vehicle_count</i>	UInt32	Broj vozila registrovanih u zoni tokom trajanja vremenskog prozora – gustina saobraćaja

Tabela 7. Struktura tabele *zone_density*

5.4. Implementacija analiza

5.4.1. Geofencing - detekcija ulaska, izlaska i zadržavanja u zoni

Osnovna prostorna operacija sistema je provera da li se neka GPS tačka nalazi unutar neke od šest definisanih zona. Svaka zona je modelovana kao krug, definisan geografskim centrom i radijusom u metrima. Provera pripadnosti zoni implementirana je korišćenjem *Haversine* formule, koja iz geografskih koordinata dve tačke (centra zone i trenutne pozicije vozila) računa udaljenost između njih po površini sfere. Ako je izračunata udaljenost manja od radijusa zone, vozilo se smatra da je unutar te zone.

```
def haversine_distance_m(lat1: float, lon1: float, lat2: float, lon2: float) -> float:

    R = 6371000 # radijus Zemlje u metrima
    phi1, phi2 = math.radians(lat1), math.radians(lat2)
    dphi = math.radians(lat2 - lat1)
    dlamba = math.radians(lon2 - lon1)
    a = (math.sin(dphi / 2) ** 2 + math.cos(phi1) * math.cos(phi2) * math.sin(dlamba / 2) **
2)
    return 2 * R * math.asin(math.sqrt(a))

def zones_containing_point(lat: float, lon: float) -> list[str]:
    result = []
    for zone in ZONES:
        distance = haversine_distance_m(lat, lon, zone["lat"], zone["lon"])
        if distance <= zone["radius_m"]:
            result.append(zone["zone_id"])
    return result
```

Prilog 1. Implementacija Haversine formule i određivanje zona kojima pripada GPS tačka

Detekcija ulaska (ENTER), izlaska (EXIT) i zadržavanja (DWELL) implementirana je u Flink-u korišćenjem *KeyedProcessFunction*, gde je tok podataka prethodno podeljen po identifikatoru vozila, čime je obezbeđeno da se za svako vozilo nezavisno održava stanje.

Teorijski je moguće da se vozilo u jednom trenutku nalazi u više zona ukoliko se zone preklapaju. Zbog toga je za čuvanje stanja izabrana *MapState* struktura, koja omogućava da se za svako vozilo održava skup svih aktivnih zona i vreme ulaska u svaku od njih. Ovakav pristup obezbeđuje da logika ostane ispravna i u slučaju naknadnog dodavanja novih zona ili promene postojećih zona tako da se one međusobno preklapaju, bez potrebe za izmenom implementacije stanja.

```
def open(self, runtime_context: RuntimeContext):
    self.current_zones_state = runtime_context.get_map_state(
        MapStateDescriptor("current_zones", Types.STRING(), Types.LONG())
    )
    self.last_dwell_emit_state = runtime_context.get_map_state(
        MapStateDescriptor("last_dwell_emit", Types.STRING(), Types.LONG())
    )
    self.last_exited_zone_state = runtime_context.get_state(
        ValueStateDescriptor("last_exited_zone", Types.STRING())
    )
```

Prilog 2. Inicijalizacija Flink stanja za praćenje zona

Za svaki novi GPS zapis koji stigne za isto vozilo, sistem upoređuje skup zona u kojima se vozilo trenutno nalazi sa skupom zona iz prethodnog zapisa. Ako se nove vrednosti latitude i longitude nalaze u zoni koje se ne nalazi u skupu, generiše se ENTER događaj za tu zonu. Za zone koje više ne pokrivaju nove vrednosti latitude i longitude, generišu se EXIT događaji sa konačnim, izračunatim trajanjem posete *dwell_seconds*, a za zone u kojima je vozilo ostalo generišu se periodični DWELL događaji ako je vreme zadržavanja prešlo definisani prag (120 sekundi), uz ograničenje da se DWELL ponavlja najviše jednom u 60 sekundi.

```
def process_element(self, value, ctx: "KeyedProcessFunction.Context"):
    ...
    entered_zones = zones_now - zones_before
    for zone_id in entered_zones:

        yield from self._emit_event(
            event_time_ms, vehicle_id, route_id, zone_id,
            "ENTER", lat, lon, dwell_seconds=None,
        )

        self.current_zones_state.put(zone_id, event_time_ms)
    ...
    # EXIT za zone u kojima je vozilo bilo pre, ali nije sada
    exited_zones = zones_before - zones_now #trajanje posete
    for zone_id in exited_zones:
```

```

        entry_time_ms = self.current_zones_state.get(zone_id)
        total_dwell_seconds = int((event_time_ms - entry_time_ms) // 1000)

        yield from self._emit_event(
            event_time_ms, vehicle_id, route_id, zone_id,
            "EXIT", lat, lon, dwell_seconds=total_dwell_seconds,
        )

        if (total_dwell_seconds >= ALERT_THRESHOLD_SECONDS
            and self.alert_emitted_state.get(zone_id) is None):
            yield from self._emit_event(
                event_time_ms, vehicle_id, route_id, zone_id,
                "ALERT", lat, lon, dwell_seconds=total_dwell_seconds,
            )

        self.current_zones_state.remove(zone_id)
        self.last_dwell_emit_state.remove(zone_id)
        self.alert_emitted_state.remove(zone_id)
        self.last_exited_zone_state.update(f"{zone_id}|{event_time_ms}")

    # DWELL: zone u kojima se vozilo nalazi
    still_in_zones = zones_now & zones_before
    for zone_id in still_in_zones:
        entry_time_ms = self.current_zones_state.get(zone_id)
        dwell_seconds = int((event_time_ms - entry_time_ms) // 1000)
        ...
        if dwell_seconds < DWELL_THRESHOLD_SECONDS:
            continue

        last_emit = self.last_dwell_emit_state.get(zone_id)
        if last_emit is not None:
            seconds_since_last_emit = (event_time_ms - last_emit) // 1000
            if seconds_since_last_emit < DWELL_REPEAT_SECONDS:
                continue # vec je emitovan DWELL za ovu zonu

        yield from self._emit_event(
            event_time_ms, vehicle_id, route_id, zone_id,
            "DWELL", lat, lon, dwell_seconds=dwell_seconds,
        )

    self.last_dwell_emit_state.put(zone_id, event_time_ms)

```

Prilog 3. Logika generisanja ENTER, EXIT i DWELL događaja

Bitna razlika u semantici između DWELL i EXIT događaja jeste da DWELL nosi trenutno, parcijalno trajanje zadržavanja, dok EXIT nosi konačno, ukupno trajanje cele posete. Za statističku analizu zadržavanja (npr. prosečno trajanje po zoni) treba da se koriste isključivo

EXIT zapisi, kako bi se izbeglo dupliranje istog zadržavanja u analizi.

Svi emitovani događaji pamte se u tabeli *zone_events*.

5.4.2. Detekcija predugog zadržavanja

Ako vreme zadržavanja vozila u zoni, računato pri proveri uslova za ispunjenje emitovanja DWELL i EXIT događaja, prvi put pređe prag od 30 minuta, sistem emituje ALERT događaj, kako bi sistem tačno jednom po poseti zoni emitovao ovaj događaj. Pored vremena ulaska u zone, za svaku aktivnu zonu čuva se i indikator da li je za trenutnu posetu već emitovan događaj ovog tipa.

```
def process_element(self, value, ctx: "KeyedProcessFunction.Context"):  
    ...  
    #ALERT emituje se prvi put kad se predje prag  
    if (dwell_seconds >= ALERT_THRESHOLD_SECONDS and self.alert_emitted_state.get(zone_id)  
        is None):  
        yield from self._emit_event(  
            event_time_ms, vehicle_id, route_id, zone_id,  
            "ALERT", lat, lon, dwell_seconds=dwell_seconds,  
        )  
        self.alert_emitted_state.put(zone_id, 1)  
    ...
```

Prilog 4. Generisanje ALERT događaja

Izabrani prag služi kao primer pravila za detekciju anomalija i može se prilagoditi konkretnim zahtevima sistema. Na ovaj način implementacija pokazuje kako se postojeća logika može proširiti dodatnim analizama zasnovanim na stanju i vremenskim uslovima, bez izmene osnovnog toka obrade podataka. U realnim sistemima isti mehanizam mogao bi da se iskoristi za prepoznavanje kašnjenja vozila, zastoja u saobraćaju, dužih zadržavanja na stanicama ili drugih operativnih problema.

ALERT događaji pamte se u tabeli *zone_events*.

5.4.3. Sekvence kretanja između zona

U okviru ovog sistema definisane *geofencing* zone međusobno su prostorno disjunktne, odnosno ne preklapaju se. U takvom modelu, sekvenca ENTER/EXIT događaja može se koristiti za rekonstrukciju kretanja vozila između zona bez potrebe za dodatnim prostornim razrešavanjem preklapanja. Ukoliko bi se u budućem proširenju sistema dozvolilo definisanje preklapajućih zona, trenutna implementacija logike za detekciju tranzicija zahtevala bi dodatne

mehanizme razrešavanja ambiguiteta. U tom slučaju bilo bi neophodno uvesti pravila prioritizacije zona, dodatnu analizu simultanih pripadnosti više zona ili model zasnovan na vremenskim sekvencama poseta, kako bi se izbegle pogrešne interpretacije prelazaka između zona.

Ova funkcionalnost je implementirana proširenjem iste *KeyedProcessFunction* funkcije. Pri poslednjem EXIT događaju svakog vozila, čuva se identifikator napuštene zone i vreme izlaska iz te zone u dodatnom Flink stanju korišćenjem strukture *ValueState*.

```
self.last_exited_zone_state = runtime_context.get_state(  
    ValueStateDescriptor("last_exited_zone", Types.STRING())  
)
```

Prilog 5. Definicija stanja za pamćenje poslednje napuštene zone

Kada to vozilo sledeći put uđe u bilo koju zonu, sistem proverava da li postoji zapamćena prethodno napuštena zona, ako postoji i ako se razlikuje od zone u koju vozilo upravo ulazi, emituje se TRANSITION događaj koji sadrži identifikatore polazne i odredišne zone, kao i vreme proteklo između izlaska i ulaska, *travel_time_seconds*.

```
def process_element(self, value, ctx: "KeyedProcessFunction.Context"):  
    ...  
  
    #TRANSITION ako postoji prethodno napustena zona  
    last_exited_raw = self.last_exited_zone_state.value()  
    if last_exited_raw is not None:  
        from_zone_id, exit_time_ms_str = last_exited_raw.split("|")  
        exit_time_ms = int(exit_time_ms_str)  
  
        if from_zone_id != zone_id:  
            travel_time_seconds = int((event_time_ms - exit_time_ms) // 1000)  
            yield from self._emit_transition(  
                event_time_ms, vehicle_id, route_id,  
                from_zone_id, zone_id, travel_time_seconds,  
            )  
            self.last_exited_zone_state.clear()  
        ...  
    self.last_exited_zone_state.update(f"{zone_id}|{event_time_ms}")
```

Prilog 6. Generisanje TRANSITION događaja između zona

Rezultati ove analize upisuju se u tabelu *zone_transitions*.

5.4.4. Agregacija broja vozila po zoni kroz vremenske prozore

Analiza broja vozila po zoni implementirana je kao samostalan Flink *job* jer demonstrira drugačiji pristup *stream processing*-u u odnosu na prethodne analize. Umesto *stateful KeyedProcessFunction*, koristi se Flink-ov ugrađeni *window API*.

Iz svakog GPS zapisa generišu se parovi (identifikator zone, identifikator vozila) za sve zone u kojima se vozilo trenutno nalazi, nakon čega se tok podataka grupiše po zoni i deli na *tumbling* vremenske prozore u trajanju od 5 minuta. Za svaki prozor i svaku zonu, sistem broji broj jedinstvenih vozila viđenih u toj zoni tokom trajanja prozora, što predstavlja meru trenutne gustine saobraćaja.

```
def main():
    ...
    density_stream = (
        zone_vehicle_pairs
        .key_by(lambda t: t[0]) # kljuc = zone_id
        .window(TumblingEventTimeWindows.of(Time.minutes(WINDOW_SIZE_MINUTES)))
        .process(CountDistinctVehiclesPerZone(), output_type=Types.STRING())
    )
    ...

class CountDistinctVehiclesPerZone(ProcessWindowFunction):
    def process(self, key: str, context: "ProcessWindowFunction.Context", elements):
        vehicle_ids = set()
        route_ids = set()

        for zone_id, vehicle_id, route_id in elements:
            vehicle_ids.add(vehicle_id)
            if route_id:
                route_ids.add(route_id)

        window_end_ms = context.window().end
        zone_name = next((z["zone_name"] for z in ZONES if z["zone_id"] == key), key)

        result = {
            "window_end_ms": window_end_ms,
            "zone_id": key,
            "zone_name": zone_name,
            "vehicle_count": len(vehicle_ids),
            "route_ids": sorted(route_ids),
        }

        yield json.dumps(result)
```

Prilog 7. Agregacija jedinstvenih vozila po zoni u petominutnom prozoru

U ovoj implementaciji korišćen je *event time* događaja sa *watermark* mehanizmom opisanog teorijski u poglavlju 3.3.3.

```
watermark_strategy = (  
    WatermarkStrategy  
        .for_bounded_out_of_orderness(Duration.of_seconds(WATERMARK_MAX_OUT_OF_ORDERNESS_SECONDS))  
        .with_timestamp_assigner(VehiclePositionTimestampAssigner())  
)
```

Prilog 8. Dodeljivanje *event time* oznaka i generisanje *watermark*-a

Rezultati agregacija za svaku zonu upisuju se u *zone_density* tabelu.

5.5. Vizualizacija rezultata

Konstruisana su četiri grafika koji prikazuju različite aspekte obrade prostorno-vremenskih podataka u realnom vremenu, a osvežavaju se automatski kako novi podaci pristižu iz Flink *job*-ova u ClickHouse.

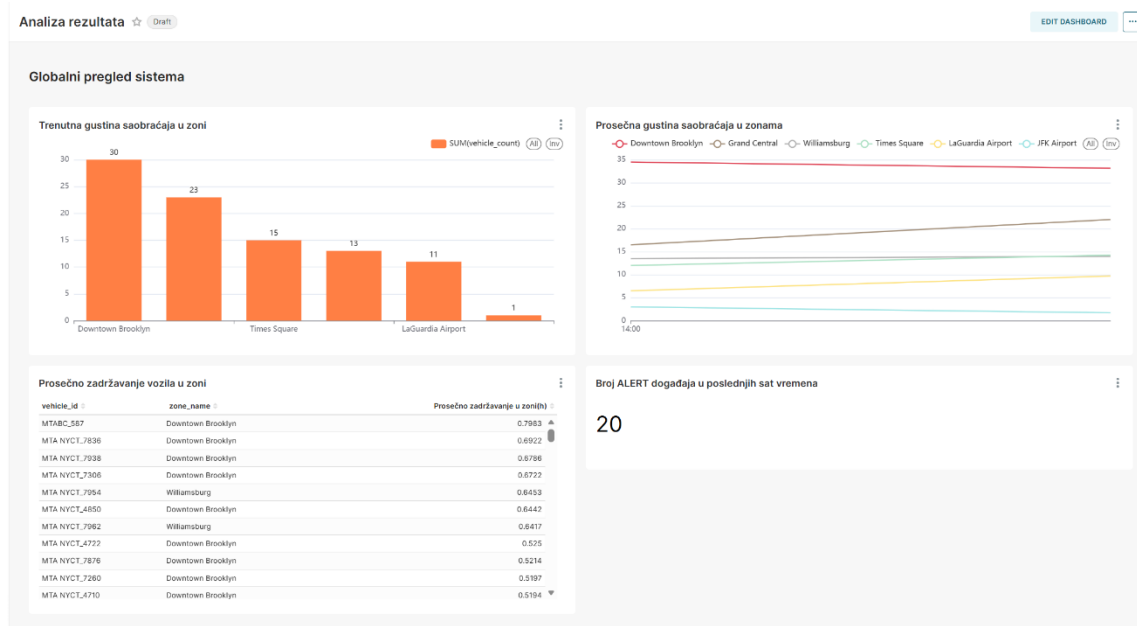
Prvi grafik prikazuje trenutnu gustinu saobraćaja po zonama, odnosno broj vozila registrovanih u svakoj zoni tokom poslednjeg završenog petominutnog vremenskog prozora. Vrednosti se računaju iz tabele *zone_density* i daju trenutni uvid u to koje zone su najopterećenije u datom trenutku. Ovaj grafik direktno reflektuje rezultate *density job*-a opisanog u poglavlju 5.4.4.

Drugi grafik prikazuje prosečnu gustinu saobraćaja po zonama kroz duži vremenski period, računatu kao prosek broja vozila po zoni kroz sve završene prozore. Za razliku od prvog grafika, koji oslikava trenutno stanje, ovaj grafik omogućava poređenje zona po njihovoj prosečnoj opterećenosti i otkriva koje zone su strukturalno frekventnije od ostalih.

Treći grafik prikazuje prosečno zadržavanje vozila u zoni, izraženo u sekundama, računato isključivo na osnovu EXIT događaja iz tabele *zone_events*. Kako je opisano u poglavlju 5.4.1, za ovu analizu se koriste EXIT zapisi koji nose konačno trajanje posete.

Četvrti grafik prikazuje ukupan broj ALERT događaja u poslednjem satu. ALERT događaji, opisani u poglavlju 5.4.2, generišu se kada vozilo provede više od 30 minuta u istoj zoni, što može ukazivati na zastoje, kvar ili neočekivano dugo zadržavanje. Ovaj grafik služi kao operativni indikator anomalija.

Slika 2 prikazuje *dashboard* sa četiri opisana grafika.



Slika 2. Vizualni prikaz rezultata u Apache SuperSet-u

5.6. Eksperimentalna evaluacija

5.6.1. Analiza uticaja paralelizma na performanse sistema

U prvoj fazi testiranja sistem je pokrenut sa jednim TaskManager kontejnerom kome je dodeljeno 2GB memorije i 8 dostupnih *task slot*-ova. Variran je nivo paralelizma Flink *job*-a kroz tri konfiguracije koje se mogu videti u tabeli 8, pri čemu je paralelizam postavljen programski putem *env.set_parallelism()* poziva, a broj Kafka particija je usklađivan sa nivoom paralelizma. Usklađivanje broja particija sa paralelizmom je neophodno jer Kafka particiju može čitati najviše jedan *task* u isto vreme, npr. pri paralelizmu 4 i samo 2 particije, dva *task*-a ne bi dobijala podatke, ali bi i dalje trošila resurse.

Važno je napomenuti da sve tri konfiguracije nisu podrazumevale pokretanje više procesa ili kontejnera, već variranje broja niti unutar istog TaskManager procesa. Sve niti su delile isti memorijski prostor od 2GB, što je, kako će rezultati pokazati, predstavljalo dominantno ograničenje sistema.

Konfiguracija	Broj TaskManager-a	Paralelizam	Broj Kafka particija	Memorija po TaskManager-u
Broj 1	1	1	1	2GB
Broj 2	1	2	2	2GB

Broj 3	1	4	4	2GB
--------	---	---	---	-----

Tabela 8. Prikaz konfiguracija za testiranje u prvoj fazi

Testiranje je sprovedeno sa oba *job*-a (*density* i *geofencing*) koja se izvršavaju istovremeno. Prilikom analize performansi fokus je stavljen na *geofencing job*, jer on predstavlja složeniji deo obrade. Zbog prirode obrade i direktne zavisnosti od količine obrađenih ulaznih zapisa, *throughput* ovog *job*-a je korišćen kao osnovna metrika performansi sistema.

Density job je pokretan paralelno kako bi se testiralo ponašanje sistema u realnom scenariju sa više istovremenih *stream processing* komponenti, ali njegove metrike nisu posebno analizirane, jer predstavlja nezavisnu vremensku agregaciju zasnovanu na prozorskom modelu i ne meri se na isti način kao *stateful geofencing* obrada.

Za svaku konfiguraciju su beleženi *throughput geofencing job*-a, CPU opterećenje Flink TaskManager kontejnera i zauzeće memorije.

Konfiguracija	Throughput (rec/s)	CPU TaskManager (%)	Memory usage TaskManager (%)
Broj 1	218	27.33	97.10
Broj 2	108	34.34	98.55
Broj 3	53	54.87	99.01

Tabela 9. Rezultati prve faze testiranja

Rezultati pokazuju da povećanje paralelizma unutar jednog TaskManager-a nije dovelo do poboljšanja performansi sistema. Najbolji *throughput* ostvaren je u konfiguraciji 1, dok su konfiguracije sa većim opterećenjem TaskManager-a imale značajno manju propusnost. Povećanje CPU iskorišćenja i memorijske zauzetosti u konfiguracijama 2 i 3 ukazuje na povećan trošak usled većeg broja izvršnih jedinica i komunikacije između komponenti. Takođe, visoka memorijska zauzetost (preko 97% u svim konfiguracijama) predstavlja potencijalno usko grlo koje može dovesti do degradacije performansi i prekida rada TaskManager-a.

Dobijeni rezultati ne predstavljaju kontradikciju teorijskim prednostima paralelne obrade, već odražavaju specifičnosti konkretnog sistema i uslova testiranja. Nekoliko faktora zajedno dovodi do ovakvog ponašanja. Pre svega, *geofencing job* koristi *stateful KeyedProcessFunction* sa stanjem vezanim za *vehicle_id*, što znači da Flink mora da garantuje da sve poruke istog vozila stignu do istog *task*-a. Pri paralelizmu većem od 1, svaka poruka

prolazi kroz *keyBy shuffle* serijalizaciju, slanje kroz Flink-ov mrežni sloj i deserijalizaciju na drugoj strani, pa taj *overhead* postaje dominantan trošak pri relativno malom intenzitetu ulaznog toka. Dodatno, MTA feed se osvežava svakih 15 sekundi i distribuira poruke po particijama na osnovu *vehicle_id* ključa, što pri većem broju particija dovodi do neravnomerne raspodele podataka između taskova, neke Flink instance čekaju na podatke dok druge završe obradu. Konačno, visoka memorijska zauzetost TaskManager kontejnera, koja u svim konfiguracijama prelazi 97% od dodeljenih 2GB, ukazuje da je memorija bila primarno usko grlo sistema već u prvoj konfiguraciji, zbog čega je Flink pri paralelizmu 2 i 4 bio primoran da deo stanja prebaci na disk, što direktno usporava pristup stanju po vozilu.

5.6.2. Analiza horizontalne skalabilnosti Flink klastera

U drugoj fazi testiranja ispituje se alternativni pristup zasnovan na horizontalnom skaliranju kroz pokretanje više TaskManager instanci. Za razliku od prethodnog eksperimenta gde su sve niti delile isti memorijski prostor, svaki TaskManager predstavlja poseban kontejner sa sopstvenom dodeljenom memorijom.

Testirane su dve konfiguracije, prikazane u tabeli 10, koje su direktno uporedive sa konfiguracijama 2 i 3 iz prethodne sekcije, zbog istog paralelizama i broja Kafka particija.

Konfiguracija	Broj TaskManager-a	Paralelizam	Broj Kafka particija	Memorija po TaskManager-u
Broj 4	2	2	2	1GB
Broj 5	4	4	4	1GB

Tabela 10. Prikaz konfiguracija za testiranje u drugoj fazi

Na osnovu rezultata koji se prikazani u tabeli 11, uočava se da *throughput* opada sa povećanjem broja TaskManager-a (sa 57 na 34 rec/s), što je sličan trend kao u sekciji 5.6.1. Međutim, CPU opterećenje je znatno niže i ravnomernije raspoređeno, prosečno oko 10% po TaskManager-u, naspram 34-55% u sekciji 5.6.1. Memorijska zauzetost je nešto niža u konfiguraciji 5 (prosek 92.5%) u poređenju sa konfiguracijama sa jednim TaskManager-om, što ukazuje da raspodela memorijskog prostora na više procesa delimično ublažava memorijsko usko grlo. Primetna je i neravnomerna raspodela CPU opterećenja između TaskManager-a.

Konfiguracija	TaskManager	Throughput (rec/s)	CPU TaskManager (%)	Memory usage TaskManager (%)
Broj 4	taskmanager-1	57	15.7	98.2
Broj 4	taskmanager-2	57	4.8	97.6
Broj 5	taskmanager-1	34	12.79	93.45
Broj 5	taskmanager-2	34	16.37	92.89
Broj 5	taskmanager-3	34	4.81	87.81
Broj 5	taskmanager-4	34	4.80	95.84

Tabela 11. Rezultati druge faze testiranja

Rezultati oba eksperimenta konzistentno pokazuju da povećanje paralelizma u ovom konkretnom testnom okruženju nije dovelo do poboljšanja *throughputa*, bez obzira na to da li je paralelizacija ostvarena kroz više niti unutar jednog TaskManager-a ili kroz više odvojenih TaskManager instanci.

6. Zaključak

U ovom radu je predstavljena analiza i implementacija sistema za obradu velikih tokova prostorno-vremenskih podataka korišćenjem Apache Flink platforme. Fokus je bio na *real-time geofencing* obradi GPS podataka vozila, pri čemu su definisane prostorne zone i implementirana logika za detekciju ulaska i izlaska vozila iz tih zona. Na osnovu osnovnog toka podataka razvijen je sistem koji omogućava generisanje više tipova izlaznih informacija, uključujući geofence događaje, statistike o zadržavanju vozila u zonama, kao i agregirane informacije o broju vozila po zoni po vremenskim prozorima.

Rezultati implementacije pokazuju da Apache Flink omogućava efikasnu obradu velikog broja GPS događaja u realnom vremenu, uz nisku latenciju i stabilnu propusnost sistema. Posebno je značajno da sistem podržava stateful obradu podataka, što omogućava izračunavanje kompleksnijih metrika kao što su vreme zadržavanja vozila u određenoj zoni i trenutna popunjenost zona. Evaluacija performansi sprovedena je kroz dve faze. U prvoj fazi varirani su nivoi paralelizma unutar jednog TaskManager-a kroz tri konfiguracije, a u drugoj fazi ispitano je horizontalno skaliranje pokretanjem više TaskManager instanci. Rezultati obe faze konzistentno pokazuju da povećanje paralelizma u konkretnom testnom okruženju nije dovelo do poboljšanja propusnosti, najbolji *throughput* od 218 rec/s ostvaren je u konfiguraciji sa paralelizmom 1 i jednim TaskManager-om, dok je svako povećanje paralelizma, bez obzira na pristup, rezultovalo padom propusnosti. Analiza ukazuje na tri glavna uzroka ovakvog ponašanja: visoka memorijska zauzetost koja u većini konfiguracija prelazi 90% i prisiljava Flink da deo stanja prebaci na disk, *shuffle overhead* koji nastaje pri *keyBy* operacijama u *stateful* obradi pri paralelizmu većem od 1, kao i neravnomerna raspodela podataka po Kafka particijama uslovljena distribucijom *vehicle_id* ključeva. Ova neravnomernost bila je posebno vidljiva u konfiguraciji sa četiri TaskManager-a kroz CPU opterećenje gde su dva TaskManager-a bila značajno aktivnija (12-16% CPU) od preostala dva (oko 5% CPU). Dobijeni rezultati ne negiraju teorijske prednosti paralelne i distribuirane obrade, već odražavaju ograničenja konkretnog testnog okruženja, pre svega nedovoljno dodeljene memorije po instanci i relativno mali intenzitet ulaznog toka. U produkcijskom scenariju sa adekvatno dimenzionisanim klasterom, većom memorijom po TaskManager-u i višestruko većim tokom podataka, očekuje se da bi i povećanje paralelizma i horizontalno skaliranje doneli merljive benefite.

Iako implementirani sistem uspešno demonstrira principe *stream* obrade prostorno-vremenskih podataka, postoje određena ograničenja. *Geofencing* logika je zasnovana na Haversine formuli za računanje udaljenosti, što predstavlja jednostavniji pristup u odnosu na upotrebu prostornih indeksa ili GIS optimizacija. Ovaj izbor je donet u kontekstu fokusiranja rada na *stream processing*, ali može predstavljati ograničenje u scenarijima sa veoma velikim brojem zona ili ekstremno visokom frekvencijom podataka. Takođe, sistem koristi unapred definisane geografske zone, bez dinamičkog prilagođavanja.

Kao pravci budućeg rada, moguće je unaprediti sistem u nekoliko pravaca. Pre svega, implementacija prostornih indeksa mogla bi poboljšati performanse *geofencing* operacija.

Dalje unapređenje može uključivati primenu naprednih metoda mašinskog učenja za detekciju anomalija i predikciju kretanja vozila. Takođe, proširenje sistema na još veće distribuisane klastere i testiranje pod ekstremnim opterećenjima omogućilo bi detaljniju analizu skalabilnosti.

U celini, rad pokazuje da Apache Flink predstavlja moćnu platformu za obradu velikih tokova prostorno-vremenskih podataka i da se efikasno može primeniti u sistemima za analizu kretanja vozila u realnom vremenu.

7. Reference

- [1] IBM, *What is Big Data?*, <https://www.ibm.com/think/topics/big-data>, datum poslednjeg pristupa: 19.07.2026.
- [2] IBM, *What is Streaming Data?*, <https://www.ibm.com/think/topics/streaming-data>, datum poslednjeg pristupa: 19.07.2026.
- [3] Apache Kafka, *Apache Kafka Documentation*, <https://kafka.apache.org/documentation/>, datum poslednjeg pristupa: 19.07.2026.
- [4] Apache Spark, *Structured Streaming Programming Guide*, <https://spark.apache.org/docs/latest/streaming/index.html>, datum poslednjeg pristupa: 19.07.2026.
- [5] Apache Flink, *Apache Flink Documentation*, <https://nightlies.apache.org/flink/flink-docs-stable/>, datum poslednjeg pristupa: 19.07.2026.
- [6] Apache Kafka, *Kafka Streams Documentation*, <https://kafka.apache.org/documentation/streams/>, datum poslednjeg pristupa: 19.07.2026.